

Detecting and locating chess pieces using Convolutional Neural Networks (CNN) and YOLO5 with Ultralytics

2025-11-17

```
!git clone https://github.com/ultralytics/yolov5 # Clone repository
%cd yolov5
!pip install -r requirements.txt # Install dependencies
```

Introduction to some Computer Vision concepts

Ziel: Schachfiguren auf einem Schachbrett erkennen

- früher: vorgegebene Objekte einfach Bildern finden
- man kann ein Bild/icons ausschneiden und dieses in einem anderen Bild finden

Nachteil - dieses Bild musste dem Image in einem anderen Bild sehr ähnlich sein (hing u.a. an einem vorgegebenen *similarity threshold* ab)

Picture with White King



Cut Image White King



```
kingscr = cv.matchTemplate(whiteking, cut_rgb ,cv.TM_CCoeff_NORMED)
threshold = 0.7
locback = np.where(kingscr >= threshold)
locback = np.array(locback)
```

```

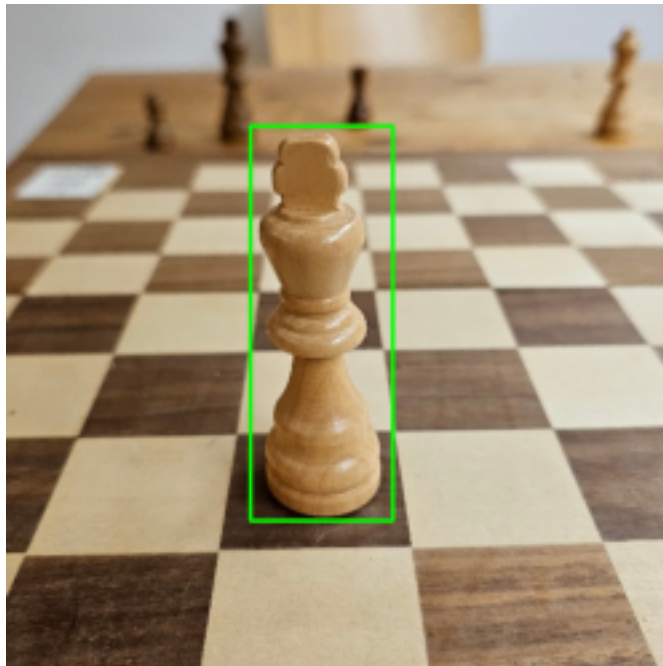
# Draw a rectangle around the detected object
color = (0, 255, 0) # Green color in BGR
thickness = 2 # Thickness of the rectangle border
cv.rectangle(whiteking, (locback[1][0], locback[0][0]), (locback[1][0] + 109, locback[0][0] + 303), color, thickness)

# Convert from BGR to RGB with openCV
img_rgb = cv.cvtColor(whiteking, cv.COLOR_BGR2RGB)

# Display the image
plt.imshow(img_rgb)
plt.axis('off') # Hide axes

plt.show()

```



```

# Convert to grayscale
gray = cv.cvtColor(whiteking1, cv.COLOR_BGR2GRAY)

# Apply Gaussian Blur to reduce noise
blurred = cv.GaussianBlur(gray, (5, 5), 0)

# Perform edge detection (Canny)
edges = cv.Canny(blurred, 50, 150)

# Find contours

```

```

contours, hierarchy = cv.findContours(edges, cv.RETR_TREE, cv.CHAIN_APPROX_SIMPLE)

whiteking1 = cv.cvtColor(whiteking1, cv.COLOR_BGR2RGB)
# Draw all contours on the original image
output = whiteking1.copy()

# Display the image
plt.imshow(cv.drawContours(output, contours, -1, (0, 255, 0), 2))
plt.axis('off') # Hide axes

plt.show()

```



Simple **Algorithmen** zur Objekterkennung - Lernen nicht - vorgegebene Objekte erkennen (müssen sehr ähnlich sein) - Erkennen, dass Objekte auf dem Bild sind -

→

Einschränkungen bei der Erkennung und Identifizierung von Objekttypen 1. CNN: hier verschiedene Schachfiguren kategorisieren 2. YOLO5, Ultralytics: mehrere verschiedene Schachfiguren auf einem Schachbrett finden und kategorisieren

CNN: Convolutional Networks

- Bilder kategorisieren
- ähnliche Bilder von einem bestimmten Objekttyp (Katze, Hund, Alpaka) in einem Foto identifizieren

Datenvorbereitung

Für den Datensatz für die CNN Kategorisierung Bilder einzelner Schachfiguren aufgenommen: - in das Format 512x512 skaliert - 12 Kategorien - für jede Schachfigur



- Funktion die alle Bilder in einem np.array zusammenführt und als Variable x zurückgibt
- y sind jeweiligen Ziellabels zu den jeweiligen Bildern

```
import os
import numpy as np
from PIL import Image

# Function to load images into a NumPy list
def load_images(image_folder):
    images = []

    for filename in os.listdir(image_folder):
        if filename.endswith('.jpg'):
            image_path = os.path.join(image_folder, filename)

            # Open image and convert to NumPy array
            img = Image.open(image_path)
            img_array = np.array(img)

            # Append the image array to the list
            images.append(img_array)
```

```

    return images
images = load_images("pics")
images = np.array(images)
x = images
y = np.loadtxt("y.txt")

```

Diese werden weiter für das CNN vorbereitet:

```

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import load_img
from tensorflow.keras.preprocessing.image import img_to_array

from sklearn.preprocessing import LabelBinarizer
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical
import io
import time
import os

# Durchführen eines Train Test Splits
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

# y-train werden one-hot encoded, da 12 Klassen kategorisiert werden sollen
lb = LabelBinarizer()
y_train = lb.fit_transform(y_train)
y_test = lb.transform(y_test)

```

Model Building

```

# Build the CNN model
model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),          # <- add this
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(12, activation='softmax'),
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

```

```

model.summary()

# Train the model
history = model.fit(X_train, y_train, epochs=20, batch_size=2, validation_data=(X_test, y_test))

y_pred = model.predict(X_test)

```

Ergebnisse des CNN Trainings:

Validation Training Results

```

Epoch 16/20
45/45 ----- 6s 123ms/step - accuracy: 0.9444 - loss: 0.1944 -
val_accuracy: 0.4783 - val_loss: 3.2331
Epoch 17/20
45/45 ----- 6s 123ms/step - accuracy: 0.9778 - loss: 0.0724 -
val_accuracy: 0.4348 - val_loss: 4.2770
Epoch 18/20
45/45 ----- 6s 123ms/step - accuracy: 1.0000 - loss: 0.0161 -
val_accuracy: 0.5217 - val_loss: 5.3982
Epoch 19/20
45/45 ----- 5s 119ms/step - accuracy: 1.0000 - loss: 0.0040 -
val_accuracy: 0.5217 - val_loss: 5.6404
Epoch 20/20
45/45 ----- 5s 120ms/step - accuracy: 1.0000 - loss: 0.0049 -
val_accuracy: 0.4783 - val_loss: 5.7287
1/1 ----- 0s 108ms/step

```

Prediction Results

```
y_true: 2 y_pred: 6
y_true: 6 y_pred: 0
y_true: 11 y_pred: 11
y_true: 11 y_pred: 11
y_true: 1 y_pred: 2
y_true: 4 y_pred: 4
y_true: 11 y_pred: 8
y_true: 8 y_pred: 6
y_true: 1 y_pred: 4
y_true: 2 y_pred: 10
```

Interpretation der Ergebnisse

Die Ergebnisse mit CNNs sind nicht zufriedenstellend. Es wird etwas gelernt, aber das Ergebnis ist für das Ziel nicht brauchbar. Das liegt sehr wahrscheinlich an der Qualität der Bilder, auf denen das CNN gelernt hat. Da bei jedem Bild ein unterschiedlicher Hintergrund präsentiert wird, wird dieser für jedes Bild mit gelernt. Hier sollten die Bilder mit einem identischen Hintergrund fotografiert werden. Das gelernte Modell scheint dann aber für den praktischen Einsatz ungeeignet, da die Bilder nur mit einem einheitlichen Hintergrund gelernt werden. Das ist im Einsatz beim Schachspiel nicht praktikabel. Der Einsatz von CNN für eine einfache Kategorisierung einzelner Schachfiguren scheint, wie hier dargestellt, keine geeignete Aufgabe zu sein.

Zusätzlich war der Datensatz mit 110 Bildern insgesamt recht klein.

YOLO5 und Ultralytics bieten die Möglichkeit mehrere Objekte in einem variablen Bild identifizieren zu können.

YOLO5 und Ultralytics

Mithilfe von Ultralytics lassen sich mehrere Objekte gleichzeitig in einem Bild finden und klassifizieren. Dafür müssen die Objekte in den Trainingsbildern vorher von Hand gelabelt werden. In diesem Projekt wurden die Bilder mit <https://makesense.ai>



Die Bilder werden in Ultralytics (hier auf Google Colab) aufgeteilt in einen Ordner *Images* und einen Ordner *Labels*. Der Ordner Images enthält Unterordner mit den Trainingsbildern in *train* und den Validationbildern in *val*. In Labels werden die Labels den jeweiligen Bildern entsprechend auch unter *train* und unter *val* gespeichert.

- Images
 - train
 - val
- Labels
 - train
 - val

yaml Datei erstellen

Es muss außerdem noch eine *yaml* Datei erstellt werden, um;

- die Pfade zu den Images und den Label Ordnern zu übergeben
- die Anzahl der Kategorien und der Kategorienauflistung mit Namen

Auf die Yaml Datei wird dann im Hauptprogramm verwiesen.

Code für Training


```

!git clone https://github.com/ultralytics/yolov5 # Clone repository
%cd yolov5
!pip install -r requirements.txt # Install dependencies

from ultralytics import YOLO
from google.colab import drive

drive.mount('/content/drive')

# Ein trainiertes Modell von YOLO5 laden
# und für Transfer-Learning verwenden
model = YOLO("yolov5s.pt")

# Das Modell auf die eigenen Bilder trainieren
# 10 epochen - mehr bringt auch nicht mehr
# batch size 10 - bessere Übersicht
# save_json: Ergebnisse in einer Json Datei ausgeben
# ansonsten wird auch automatisch eine .csv Datei erstellt

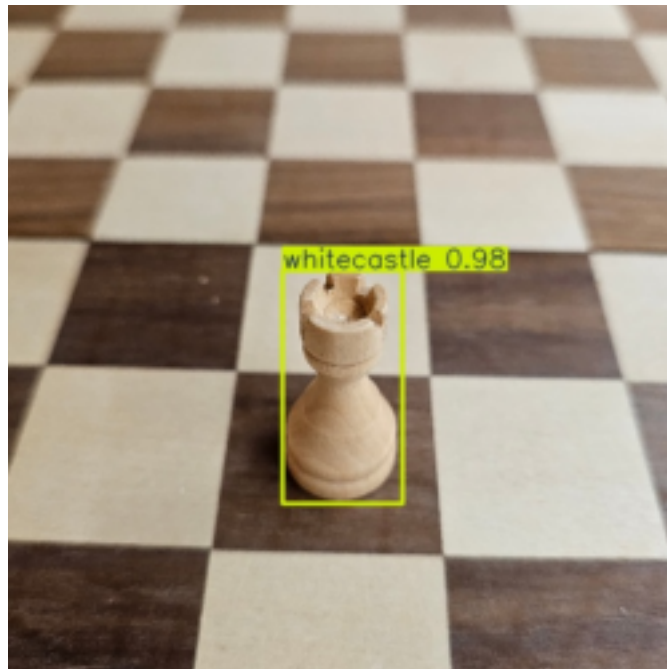
results = model.train(data="ChessPieces.yaml",
                      epochs=10,
                      imgsz = 256,
                      batch = 10,
                      save_json = True,
                      lr0=0.05,
                      augment=True,
                      flipud=0.5,
                      fliplr=0.5,
                      degrees=5,
                      scale=0.1
)

# check prediction of model for white rook in data set
result = model.predict(r"images/val/whitecastle2.jpg",
                      save=True,
                      imgsz = 256,
                      conf=0.45, # Schwellenwert 1 nur Objekte mit höchster Wahrscheinlichkeit über di
                      iou=0.5,  # Schwellenwert 2 Objekte die sich mit IOU größer diesem Wert überlapp
                      )

```

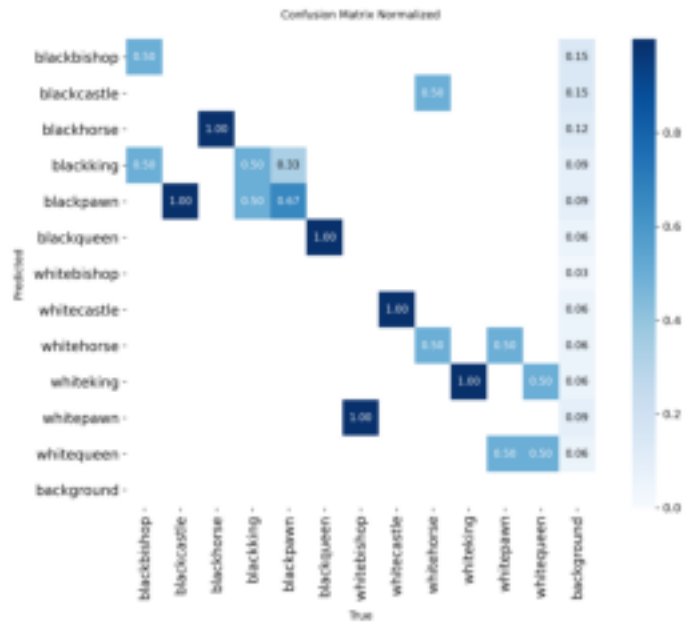
Predict Ergebnis des einen Beispiels

- Es wurde geprüft, ob ein weißer Turm erkannt wird. Zusätzlich gibt das System zurück, wie sicher es in seiner Kategorisierung ist, die richtige Entscheidung getroffen zu haben.



Confusion Matrix

- Anzeige einer normalisierten Confusion Matrix der Ergebnisse
- einige wenige Schachfiguren werden gut erkannt
- andere gar nicht sicher



Training Pictures augmented

Im Trainingsset wurden die Bilder *augmented* also mit verschiedenen variablen Verzerrungen versehen (z.B. verschoben, gedreht)



Evaluation Pictures

Hier ist das Ergebnis, wie die Schachfiguren in verschiedenen Bildern erkannt werden.



- Die Erkennung ist noch nicht besonders gut. Wahrscheinlich zu viel Rauschen durch das Fotografieren einiger Schachfiguren von oben.
- → Bilder, die von oben fotografiert wurden, sollten erstmal nicht in das Modell einfließen
- Außerdem ist das aufgenommene Dataset noch immer zu klein, um zuverlässige Ergebnisse zu liefern.
 - 120 Bilder insgesamt
- → mit deutlich mehr Bildern trainieren, um die Variabilität von den unterschiedlichen Perspektiven auszugleichen

What next?

- Mehr Bilder machen
- Bilder, die von oben fotografiert sind, erstmal rausnehmen

Großes Ziel, viele Herausforderungen

- Schachbrett separat trainieren?
- Wie werden Positionen der Felder ermittelt?
- Positionen der Figuren auf dem Schachbrett ermitteln

Mögliche Ideen

- Felder auf dem gesamten Schachbrett mit openNCV erkennen

- Felder benennen - Koordinaten bestimmen
- Diese dann mit Figuren trainieren
- Fehlende Bereiche des Schachbretts füllen - z.B. mit Generative Adversarial Networks (GANs)